

TYPE THEORY AS A TOPIC OF MATHEMATICAL STUDY

AN INTRODUCTION TO GENERALISED ALGEBRAIC THEORIES

Formal proof and synthetic mathematics workshop – June 26th 2026

Meven LENNON-BERTRAND

Inria – IRIF, Université Paris Cité

We've been doing quite a bit of type theory...
Or, to be precise, worked *in* type theories.

We've been doing quite a bit of type theory...
Or, to be precise, worked *in* type theories.

But:

- What is a type theory?
- What is a model of such a type theory?
- What does it mean “to work in” such a type theory?

We've been doing quite a bit of type theory...
Or, to be precise, worked *in* type theories.

But:

- What is a type theory?
- What is a model of such a type theory?
- What does it mean “to work in” such a type theory?

These questions can be given *mathematical* answers

WARMUP: UNIVERSAL ALGEBRA

Who here has been exposed to **universal algebra**?

Ambient logic = set-theoretic notation (but could be a type theory!): $\in$, Set , $\prod$, $\sum$...

Ambient logic = set-theoretic notation (but could be a type theory!): $\in$, Set , $\prod$, Σ ...

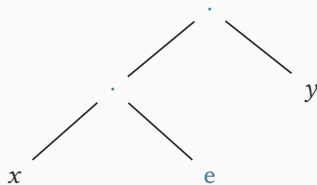
Signature

A signature Σ consists of:

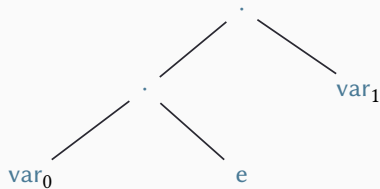
- a set of operations (generators) op_Σ , specified by an arity $\text{ar}_\Sigma \in \text{op}_\Sigma \rightarrow \mathbb{N}$

Signature of monoids :

- $\text{op} := \{e, \cdot\}$, $\text{ar}(e) := 0$, $\text{ar}(\cdot) := 2$



- They exist!
- Nice representation in (functional) programming languages (algebraic datatypes)



- They exist!
- Nice representation in (functional) programming languages (algebraic datatypes)
- Variables complicate things, but there are ways

Ambient logic = set-theoretic notation (but could be a type theory!): $\in$, **Set**, $\prod$, Σ ...

Signature

A signature Σ consists of:

- a set of operations (generators) op_Σ , specified by an arity $ar_\Sigma \in op_\Sigma \rightarrow \mathbb{N}$
→ generates a set of terms $Tm \in \mathbb{N} \rightarrow \mathbf{Set}$

Signature of monoids :

- $op := \{e, \cdot\}$, $ar(e) := 0$, $ar(\cdot) := 2$
- $e \in Tm(0)$, $(x \cdot e) \cdot y \in Tm(2)$

Ambient logic = set-theoretic notation (but could be a type theory!): $\in$, Set , $\prod$, $\sum$...

Signature

A signature Σ consists of:

- a set of operations (generators) op_Σ , specified by an arity $\text{ar}_\Sigma \in \text{op}_\Sigma \rightarrow \mathbb{N} \rightarrow \text{Set}$
→ generates a set of terms $\text{Tm} \in \mathbb{N} \rightarrow \text{Set}$
- for each n , a set of equations (relations) $\text{eqs}_\Sigma(n) \subseteq \text{Tm}(n) \times \text{Tm}(n)$

Signature of monoids :

- $\text{op} := \{e, \cdot\}$, $\text{ar}(e) := 0$, $\text{ar}(\cdot) := 2$
- $e \in \text{Tm}(0)$, $(x \cdot e) \cdot y \in \text{Tm}(2)$
- $\text{eqs}(1) := \{(x \cdot e = x), (e \cdot x = x)\}$, $\text{eqs}(3) := \{x \cdot (y \cdot z) = (x \cdot y) \cdot z\}$

Model of a signature (sort, op, ar)

- a set X

A monoid = a model of the signature of monoids:

- a set X

Model of a signature (sort, op, ar)

- a set X
- for every operation $o \in \text{op}$ with a function $f_o \in X^{\text{ar}(o)} \rightarrow X$

A monoid = a model of the signature of monoids:

- a set X
- a constant $e \in X$ and a function $f \in X \times X \rightarrow X$

Model of a signature (sort, op, ar)

- a set X
- for every operation $o \in \text{op}$ with a function $f_o \in X^{\text{ar}(o)} \rightarrow X$
→ lifted to $\llbracket _ \rrbracket \in \text{Tm}(n) \rightarrow X^n \rightarrow X$

A monoid = a model of the signature of monoids:

- a set X
- a constant $e \in X$ and a function $f \in X \times X \rightarrow X$
- $\llbracket e \rrbracket = e, \quad \llbracket (x \cdot e) \cdot y \rrbracket(u, v) = f(f(u, e), v)$

Model of a signature (sort , op , ar)

- a set X
- for every operation $\text{o} \in \text{op}$ with a function $f_{\text{o}} \in X^{\text{ar}(\text{o})} \rightarrow X$
→ lifted to $\llbracket _ \rrbracket \in \text{Tm}(n) \rightarrow X^n \rightarrow X$
- such that for every $(l, r) \in \text{eqs}(n)$ and $v \in X^n$, we have $\llbracket l \rrbracket(v) = \llbracket r \rrbracket(v)$

A monoid = a model of the signature of monoids:

- a set X
- a constant $e \in X$ and a function $f \in X \times X \rightarrow X$
- $\llbracket e \rrbracket = e$, $\llbracket (x \cdot e) \cdot y \rrbracket(u, v) = f(f(u, e), v)$
- such that for all u, v and w we have $f(e, v) = v$, $f(v, e) = v$ and $f(u, f(v, w)) = f(f(u, v), w)$

Model of a signature (sort, op, ar)

- a set X
- for every operation $o \in \text{op}$ with a function $f_o \in X^{\text{ar}(o)} \rightarrow X$
→ lifted to $\llbracket _ \rrbracket \in \text{Tm}(n) \rightarrow X^n \rightarrow X$
- such that for every $(l, r) \in \text{eqs}(n)$ and $v \in X^n$, we have $\llbracket l \rrbracket(v) = \llbracket r \rrbracket(v)$

A monoid = a model of the signature of monoids:

- a set X
- a constant $e \in X$ and a function $f \in X \times X \rightarrow X$
- $\llbracket e \rrbracket = e$, $\llbracket (x \cdot e) \cdot y \rrbracket(u, v) = f(f(u, e), v)$
- such that for all u, v and w we have $f(e, v) = v$, $f(v, e) = v$ and $f(u, f(v, w)) = f(f(u, v), w)$

+ **Morphism of model**

Theorem (originally, Birkhoff?)

Every signature has a **free model**, which is initial amongst all models.

Theorem (originally, Birkhoff?)

Every signature has a **free model**, which is initial amongst all models.

Free model for the signature of monoids	=	free monoid = $\{e\}$
Free model for the signature of monoids + 1 constant per $a \in A$	=	free monoid on A

GENERALISED ALGEBRAIC THEORY

MULTI-SORTED UNIVERSAL ALGEBRA

Monoids, rings, propositional logic... fit in universal algebra

Modules, vector spaces... need multiple sorts

Monoids, rings, propositional logic... fit in universal algebra
Modules, vector spaces... need multiple sorts

Multi-sorted signature

A signature Σ consists of:

- a **set of sorts** $\text{sort}_{\Sigma} \in \text{Set}$
- a set of operations op_{Σ} , specified by an arity $\text{ar}_{\Sigma} \in \text{op}_{\Sigma} \rightarrow \text{sort}_{\Sigma}^* \times \text{sort}_{\Sigma} \rightarrow \text{Set}$
→ generates a set of terms $\text{Tm} \in \text{sort}_{\Sigma}^* \times \text{sort}_{\Sigma} \rightarrow \text{Set}$
- a set of equations $\text{eqs}_{\Sigma}(v, s) \subseteq \text{Tm}(v, s) \times \text{Tm}(v, s)$

Monoids, rings, propositional logic... fit in universal algebra

Modules, vector spaces... need multiple sorts

- two sorts = s for scalars and v for vectors
- operations $ar(\cdot_v) := ([s, v], v)$ $ar(\cdot_s) := ([s, s], s)$ $ar(+_s) := ([s, s], s)$ $ar(+_v) := ([v, v], v)$
- + equations

Multi-sorted signature

A signature Σ consists of:

- a **set of sorts** $sort_\Sigma \in \mathbf{Set}$
- a set of operations op_Σ , specified by an arity $ar_\Sigma \in op_\Sigma \rightarrow sort_\Sigma^* \times sort_\Sigma$
→ generates a set of terms $Tm \in sort_\Sigma^* \times sort_\Sigma \rightarrow \mathbf{Set}$
- a set of equations $eqs_\Sigma(v, s) \subseteq Tm(v, s) \times Tm(v, s)$

STILL NOT ENOUGH!

But there's more!

- Category: set of morphisms $\text{Hom}(X, Y)$ for objects X, Y
- First-order logic: expressions, formulas over a given context
- Type theory: types over a context, terms of a type in a context

STILL NOT ENOUGH!

But there's more!

- Category: set of morphisms $\text{Hom}(X, Y)$ for objects X, Y
- First-order logic: expressions, formulas over a given context
- Type theory: types over a context, terms of a type in a context

Missing ingredient = dependent sorts

A category:

- sorts $\text{ob} \in \mathbf{Set}$ and $\text{hom} \in \prod_{x,y \in \text{ob}} \mathbf{Set}$
- operations $\text{id} \in \prod_{x \in \text{ob}} \text{hom}(x, x)$ and $\circ \in \prod_{g \in \text{hom}(y,z), f \in \text{hom}(x,y)} \text{hom}(x, z)$
- equations $\prod_{f \in \text{hom}(x,y)} f \circ \text{id}(x) = f$ etc.

A category:

- sorts $\text{ob} \in \mathbf{Set}$ and $\text{hom} \in \prod_{x,y \in \text{ob}} \mathbf{Set}$
- operations $\text{id} \in \prod_{x \in \text{ob}} \text{hom}(x, x)$ and $\circ \in \prod_{g \in \text{hom}(y,z), f \in \text{hom}(x,y)} \text{hom}(x, z)$
- equations $\prod_{f \in \text{hom}(x,y)} f \circ \text{id}(x) = f$ etc.

A (model of) dependent type theory:

- $\text{Ctx} \in \mathbf{Set}$ (contexts)
- $\text{Ty} \in \text{Ctx} \rightarrow \mathbf{Set}$ (types)
- $\text{Tm} \in \prod_{\Gamma \in \text{Ctx}} \text{Ty}(\Gamma) \rightarrow \mathbf{Set}$ (terms)
- ...

Use a **type theory** (= the **logical framework**) to define signatures

Use a **type theory** (= the **logical framework**) to define signatures

Signature, take 2

A signature is a context in the type theory below.

$$\frac{}{\Sigma \vdash \text{Sort}} \qquad \frac{\Sigma \vdash X : \text{Sort}}{\Sigma \vdash \text{El}(X)}$$
$$\frac{\Sigma \vdash A : \text{Sort} \quad \Sigma \vdash l : \text{El}(A) \quad \Sigma \vdash r : \text{El}(A)}{\Sigma \vdash l \equiv_A r} \quad (\text{extensional})$$

$$\Sigma, (X : \text{Sort})$$

$$\Sigma', (x : \text{El}(X))$$

$$\Sigma'', (e : x \equiv_A y)$$

Use a **type theory** (= the **logical framework**) to define signatures

Signature, take 2

A signature is a context in the type theory below.

$$\begin{array}{c}
 \frac{}{\Sigma \vdash \text{Sort}} \qquad \frac{\Sigma \vdash X : \text{Sort}}{\Sigma \vdash \text{El}(X)} \\
 \\
 \frac{\Sigma \vdash A : \text{Sort} \quad \Sigma \vdash l : \text{El}(A) \quad \Sigma \vdash r : \text{El}(A)}{\Sigma \vdash l \equiv_A r} \qquad \text{(extensional)} \\
 \\
 \frac{\Sigma \vdash A : \text{Sort} \quad \Sigma, (x : A) \vdash B}{\vdash (x : A) \rightarrow B} \qquad \frac{\Sigma \vdash f : (x : A) \rightarrow B \quad \Sigma \vdash u : \text{El}(A)}{\Sigma \vdash f u : \text{El}(B[u])} \\
 \\
 \Sigma, (X : \text{Sort}) \qquad \Sigma', (f : \text{El}(X) \rightarrow \text{El}(X)) \qquad \Sigma'', (e : (x : \text{El}(X)) \rightarrow f(f x) \equiv_X x)
 \end{array}$$

Use a **type theory** (= the **logical framework**) to define signatures

Signature, take 2

A signature is a context in the type theory below.

$$\begin{array}{c}
 \frac{}{\Sigma \vdash \text{Sort}} \qquad \frac{\Sigma \vdash X : \text{Sort}}{\Sigma \vdash \text{El}(X)} \\
 \\
 \frac{\Sigma \vdash A : \text{Sort} \quad \Sigma \vdash l : \text{El}(A) \quad \Sigma \vdash r : \text{El}(A)}{\Sigma \vdash l \equiv_A r} \qquad \text{(extensional)} \\
 \\
 \frac{\Sigma \vdash A : \text{Sort} \quad \Sigma, (x:A) \vdash B}{\vdash (x:A) \rightarrow B} \qquad \frac{\Sigma \vdash f : (x:A) \rightarrow B \quad \Sigma \vdash u : \text{El}(A)}{\Sigma \vdash f u : \text{El}(B[u])}
 \end{array}$$

For now: work with this type theory informally

THE GAT OF CATEGORIES, VERY FORMALLY

ob : Sort

hom : (x: ob) → (y: ob) → Sort

id : (x: ob) → El(hom(x, x))

comp : (x, y, z: ob) → (f: hom(x, y)) → (g: hom(y, z)) → El(hom(x, z))

idl : (x, y: ob) → (f: hom(x, y)) → comp(x, x, y, id(x), f) $\equiv_{\text{El}(\text{hom}(x, y))}$ f

idr : (x, y: ob) → (f: hom(x, y)) → comp(x, y, y, f, id(y)) $\equiv_{\text{El}(\text{hom}(x, y))}$ f

assoc : (w, x, y, z: ob) → (f: hom(w, x)) → (g: hom(x, y)) → (h: hom(y, z)) →
comp(x, y, w, f, comp(y, z, w, g, h)) $\equiv_{\text{El}(\text{hom}(w, z))}$ comp(x, z, w, comp(w, x, y, f, g), h)

THE GAT OF CATEGORIES, VERY FORMALLY

ob : Sort

hom : (x: ob) → (y: ob) → Sort

id : (x: ob) → El(hom(x, x))

comp : (x, y, z: ob) → (f: hom(x, y)) → (g: hom(y, z)) → El(hom(x, z))

idl : (x, y: ob) → (f: hom(x, y)) → comp(x, x, y, id(x), f) $\equiv_{\text{El}(\text{hom}(x, y))}$ f

idr : (x, y: ob) → (f: hom(x, y)) → comp(x, y, y, f, id(y)) $\equiv_{\text{El}(\text{hom}(x, y))}$ f

assoc : (w, x, y, z: ob) → (f: hom(w, x)) → (g: hom(x, y)) → (h: hom(y, z)) →
comp(x, y, w, f, comp(y, z, w, g, h)) $\equiv_{\text{El}(\text{hom}(w, z))}$ comp(x, z, w, comp(w, x, y, f, g), h)

What a mouthful!

THE GAT OF CATEGORIES, VERY FORMALLY

$\text{ob} : \text{Sort}$
 $\text{hom} : (x:\text{ob}) \rightarrow (y:\text{ob}) \rightarrow \text{Sort}$

$\text{id} : \{x:\text{ob}\} \rightarrow \text{El}(\text{hom}(x, x))$
 $\circ : \{x, y, z:\text{ob}\} \rightarrow (g:\text{hom}(y, z)) \rightarrow (f:\text{hom}(x, y)) \rightarrow \text{El}(\text{hom}(x, z))$

$(x, y:\text{ob}) \rightarrow (f:\text{hom}(x, y)) \rightarrow f \circ \text{id}_x \equiv f$
 $(x, y:\text{ob}) \rightarrow (f:\text{hom}(x, y)) \rightarrow \text{id}_y \circ f \equiv f$
 $(w, x, y, z:\text{ob}) \rightarrow (f:\text{hom}(w, x)) \rightarrow (g:\text{hom}(x, y)) \rightarrow (h:\text{hom}(y, z)) \rightarrow$
 $(h \circ g) \circ f \equiv h \circ (g \circ f)$

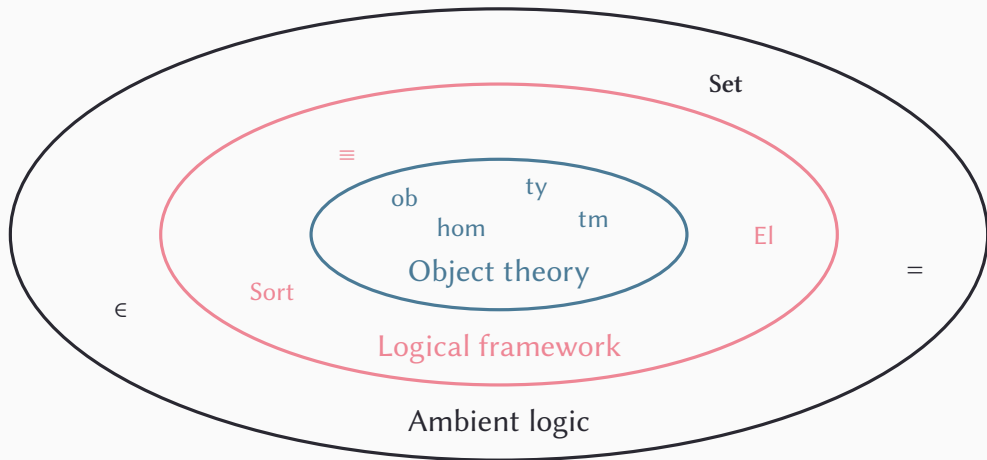
What a mouthful! Conventions: implicit arguments + notations

THE GAT OF CATEGORIES, VERY FORMALLY

$$\begin{array}{c}
 \frac{}{\vdash \text{ob}} \qquad \frac{\vdash x : \text{ob} \quad \vdash y : \text{ob}}{\vdash \text{hom}(x, y)} \\
 \\
 \frac{\vdash x : \text{ob}}{\vdash \text{id}_x : \text{hom}(x, y)} \qquad \frac{\vdash x, y, z : \text{ob} \quad \vdash f : \text{hom}(x, y) \quad \vdash g : \text{hom}(y, z)}{\vdash g \circ f : \text{hom}(x, z)} \\
 \\
 \frac{\vdash x, y : \text{ob} \quad \vdash f : \text{hom}(x, y)}{\vdash f \circ \text{id}_x \equiv f : \text{hom}(x, y)} \qquad \frac{\vdash x, y : \text{ob} \quad \vdash f : \text{hom}(x, y)}{\vdash \text{id}_y \circ f \equiv f : \text{hom}(x, y)} \\
 \\
 \frac{\vdash w, x, y, z : \text{ob} \quad \vdash f : \text{hom}(w, x) \quad \vdash g : \text{hom}(x, y) \quad \vdash h : \text{hom}(y, z)}{\vdash (h \circ g) \circ f \equiv h \circ (g \circ f) : \text{hom}(w, z)}
 \end{array}$$

What a mouthful! As typing rules

THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES



THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES

$\text{ctx} : \text{Sort}$

$\text{sub} : (\Gamma : \text{ctx}) \rightarrow (\Delta : \text{ctx}) \rightarrow \text{Sort}$

...

ctx and sub form a category

THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES

$\text{ctx} : \text{Sort}$

$\text{sub} : (\Gamma : \text{ctx}) \rightarrow (\Delta : \text{ctx}) \rightarrow \text{Sort}$

...

ctx and sub form a category

$\text{ty} : (\Gamma : \text{ctx}) \rightarrow \text{Sort}$

$_[_] : \{\Gamma, \Delta : \text{ctx}\} \rightarrow (X : \text{ty}(\Delta)) \rightarrow (\sigma : \text{sub}(\Gamma, \Delta)) \rightarrow \text{El}(\text{ty}(\Gamma))$

$(\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow X[\text{id}] \equiv X$

...

ty is a functor

THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES

$\text{ctx} : \text{Sort}$

$\text{sub} : (\Gamma : \text{ctx}) \rightarrow (\Delta : \text{ctx}) \rightarrow \text{Sort}$

...

ctx and sub form a category

$\text{ty} : (\Gamma : \text{ctx}) \rightarrow \text{Sort}$

$_[_] : \{\Gamma, \Delta : \text{ctx}\} \rightarrow (X : \text{ty}(\Delta)) \rightarrow (\sigma : \text{sub}(\Gamma, \Delta)) \rightarrow \text{El}(\text{ty}(\Gamma))$

$(\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow X[\text{id}] \equiv X$

...

ty is a functor

$\text{tm} : (\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow \text{Sort}$

$_[_] : \{\Gamma, \Delta : \text{ctx}\} \rightarrow \{X : \text{ty}(\Delta)\} \rightarrow (x : \text{tm}(\Delta, X)) \rightarrow (\sigma : \text{sub}(\Gamma, \Delta)) \rightarrow \text{El}(\text{tm}(\Gamma, X[\sigma]))$

$(\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow (x : \text{tm}(\Gamma, X)) \rightarrow x[\text{id}] \equiv_{\text{tm}(\Gamma, X)} x$

...

tm is a displayed functor

THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES

$\text{ctx} : \text{Sort}$

$\text{sub} : (\Gamma : \text{ctx}) \rightarrow (\Delta : \text{ctx}) \rightarrow \text{Sort}$

...

ctx and sub form a category

$\text{ty} : (\Gamma : \text{ctx}) \rightarrow \text{Sort}$

$_[_] : \{\Gamma, \Delta : \text{ctx}\} \rightarrow (X : \text{ty}(\Delta)) \rightarrow (\sigma : \text{sub}(\Gamma, \Delta)) \rightarrow \text{El}(\text{ty}(\Gamma))$

$(\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow X[\text{id}] \equiv X$

ty is a functor

...

$\text{tm} : (\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow \text{Sort}$

$_[_] : \{\Gamma, \Delta : \text{ctx}\} \rightarrow$

$\text{El}(\text{tr}$

$(\Gamma : \text{ctx})$

...

$(\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow (x : \text{tm}(\Gamma, X)) \rightarrow x[\text{id}] \equiv_{\text{tm}(\Gamma, X)} x$

functor

THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES (CTD.)

$\diamond : \text{El}(\text{ctx})$

$! : \{\Gamma : \text{ctx}\} \rightarrow \text{El}(\text{sub}(\Gamma, \diamond))$

$(\Gamma : \text{ctx}) \rightarrow (\sigma : \text{sub}(\Gamma, \diamond)) \rightarrow \sigma \equiv !$

ctx has a terminal object (empty context)

THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES (CTD.)

$\diamond : \text{El}(\text{ctx})$

$! : \{\Gamma : \text{ctx}\} \rightarrow \text{El}(\text{sub}(\Gamma, \diamond))$

$(\Gamma : \text{ctx}) \rightarrow (\sigma : \text{sub}(\Gamma, \diamond)) \rightarrow \sigma \equiv !$

ctx has a terminal object (empty context)

$_ \triangleright _ : (\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow \text{El}(\text{ctx})$

context extension $(\Gamma, x : X)$

THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES (CTD.)

$\diamond : \text{El}(\text{ctx})$

$! : \{\Gamma : \text{ctx}\} \rightarrow \text{El}(\text{sub}(\Gamma, \diamond))$

ctx has a terminal object (empty context)

$(\Gamma : \text{ctx}) \rightarrow (\sigma : \text{sub}(\Gamma, \diamond)) \rightarrow \sigma \equiv !$

$_ \triangleright _ : (\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow \text{El}(\text{ctx})$

context extension $(\Gamma, x : X)$

$\text{var} : \{\Gamma : \text{ctx}\} \rightarrow \{X : \text{ty}(\Gamma)\} \rightarrow \text{El}(\text{tm}(\Gamma \triangleright X, X))$ last variable in context

THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES (CTD.)

$\diamond : \text{El}(\text{ctx})$

$! : \{\Gamma : \text{ctx}\} \rightarrow \text{El}(\text{sub}(\Gamma, \diamond))$

ctx has a terminal object (empty context)

$(\Gamma : \text{ctx}) \rightarrow (\sigma : \text{sub}(\Gamma, \diamond)) \rightarrow \sigma \equiv !$

$_ \triangleright _ : (\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow \text{El}(\text{ctx})$

context extension $(\Gamma, x : X)$

$\text{var} : \{\Gamma : \text{ctx}\} \rightarrow \{X : \text{ty}(\Gamma)\} \rightarrow \text{El}(\text{tm}(\Gamma \triangleright X, X[\uparrow]))$

last variable in context

$\uparrow : \{\Gamma : \text{ctx}\} \rightarrow \{X : \text{ty}(\Gamma)\} \rightarrow \text{El}(\text{sub}(\Gamma \triangleright X, \Gamma))$

weakening

THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES (CTD.)

$\diamond : \text{El}(\text{ctx})$

$! : \{\Gamma : \text{ctx}\} \rightarrow \text{El}(\text{sub}(\Gamma, \diamond))$

ctx has a terminal object (empty context)

$(\Gamma : \text{ctx}) \rightarrow (\sigma : \text{sub}(\Gamma, \diamond)) \rightarrow \sigma \equiv !$

$_ \triangleright _ : (\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow \text{El}(\text{ctx})$

context extension $(\Gamma, x : X)$

$\text{var} : \{\Gamma : \text{ctx}\} \rightarrow \{X : \text{ty}(\Gamma)\} \rightarrow \text{El}(\text{tm}(\Gamma \triangleright X, X[\uparrow]))$

last variable in context

$\uparrow : \{\Gamma : \text{ctx}\} \rightarrow \{X : \text{ty}(\Gamma)\} \rightarrow \text{El}(\text{sub}(\Gamma \triangleright X, \Gamma))$

weakening

$(_, _) : \{\Gamma, \Delta : \text{ctx}\} \rightarrow \{X : \text{ty}(\Gamma)\} \rightarrow (\sigma : \text{sub}(\Delta, \Gamma)) \rightarrow$
 $(x : \text{tm}(\Delta, X[\sigma])) \rightarrow \text{El}(\text{sub}(\Delta, \Gamma \triangleright X))$

substitution extension

+ equations

THE TYPE THEORIST'S FAVOURITE GAT: CATEGORIES WITH FAMILIES (CTD.)

$\diamond : \text{El}(\text{ctx})$

$! : \{\Gamma : \text{ctx}\} \rightarrow \text{El}(\text{sub}(\Gamma, \diamond))$

ctx has a terminal object (empty context)

$(\Gamma : \text{ctx}) \rightarrow (\sigma : \text{sub}(\Gamma, \diamond)) \rightarrow \sigma \equiv !$

$_ \triangleright _ : (\Gamma : \text{ctx}) \rightarrow (X : \text{ty}(\Gamma)) \rightarrow \text{El}(\text{ctx})$

context extension $(\Gamma, x : X)$

$\text{var} : \{\Gamma : \text{ctx}\} \rightarrow \{X : \text{ty}(\Gamma)\} \rightarrow \text{El}(\text{tm}(\Gamma \triangleright X, X[\uparrow]))$

last variable in context

$\uparrow : \{\Gamma : \text{ctx}\} \rightarrow \{X : \text{ty}(\Gamma)\} \rightarrow \text{El}(\text{sub}(\Gamma \triangleright X, \Gamma))$

weakening

$(_, _) : \{\Gamma, \Delta : \text{ctx}\} \rightarrow \{X : \text{ty}(\Gamma)\} \rightarrow (\sigma : \text{sub}(\Delta, \Gamma)) \rightarrow$

substitution extension

$(x : \text{tm}(\Delta, X[\sigma])) \rightarrow \text{El}(\text{sub}(\Delta, \Gamma \triangleright X))$

+ equations

Only terms: $\text{var}[\uparrow^n]$ = n -th variable in context

MODELS OF A GAT

The logical framework looks a lot like the ambient logic...

The logical framework looks a lot like the ambient logic...
We can use that to interpret it! ($\approx$ Tarski semantics...)

THE STANDARD MODEL

The logical framework looks a lot like the ambient logic...
We can use that to interpret it! ($\approx$ Tarski semantics...)

	$\llbracket _ \rrbracket$	
Sort	$\mapsto$	Set
El(X)	$\mapsto$	$\llbracket X \rrbracket$
$\equiv$	$\mapsto$	$=$
$(_)\rightarrow _$	$\mapsto$	$\prod$
$_ \rightarrow _$	$\mapsto$	$\sum$

THE STANDARD MODEL

The logical framework looks a lot like the ambient logic...
We can use that to interpret it! ($\approx$ Tarski semantics...)

	$\llbracket _ \rrbracket$
Sort	$\mapsto$ Set
El(X)	$\mapsto \llbracket X \rrbracket$
$\equiv$	$\mapsto =$
$(_)\rightarrow _$	$\mapsto \prod$
$_ \rightarrow _$	$\mapsto \sum$

Definition

A **model** of a GAT Σ is an inhabitant of $\llbracket \Sigma \rrbracket$.

THE STANDARD MODEL

The logical framework looks a lot like the ambient logic...
We can use that to interpret it! ($\approx$ Tarski semantics...)

	$\llbracket _ \rrbracket$	
Sort	$\mapsto$	Set
El(X)	$\mapsto$	$\llbracket X \rrbracket$
$\equiv$	$\mapsto$	$=$
$(_) \rightarrow _$	$\mapsto$	$\prod$
$_ \rightarrow _$	$\mapsto$	$\sum$

Definition

A **model** of a GAT Σ is an inhabitant of $\llbracket \Sigma \rrbracket$.

In fact, we have a **category** $\text{Mod}(\Sigma)$ (might be too strict to your liking!)

WHAT IS A CwF?

Read from the GAT definition! Or...

WHAT IS A CwF?

Read from the GAT definition! Or...

Definition (Fiore 2012, Awodey 2018)

A **natural model** consists of:

- a category $\mathcal{C}$ with a terminal object = contexts + substitutions + empty context
- two presheaves Ty and Tm (functors $\mathcal{C}^{op} \rightarrow \mathbf{Set}$) = types, terms, action of substitutions
- a natural transformation $ty : Tm \rightarrow Ty$ = maps a term to its type

WHAT IS A CwF?

Read from the GAT definition! Or...

Definition (Fiore 2012, Awodey 2018)

A **natural model** consists of:

- a category $\mathcal{C}$ with a terminal object = contexts + substitutions + empty context
- two presheaves Ty and Tm (functors $\mathcal{C}^{op} \rightarrow \mathbf{Set}$) = types, terms, action of substitutions
- a natural transformation $ty : Tm \rightarrow Ty$ = maps a term to its type
- such that ty is locally representable = variables

WHAT IS A CwF?

Read from the GAT definition! Or...

Definition (Fiore 2012, Awodey 2018)

A **natural model** consists of:

- a category $\mathcal{C}$ with a terminal object = contexts + substitutions + empty context
- two presheaves Ty and Tm (functors $\mathcal{C}^{op} \rightarrow \mathbf{Set}$) = types, terms, action of substitutions
- a natural transformation $ty : Tm \rightarrow Ty$ = maps a term to its type
- such that ty is locally representable = variables

“ $\Gamma \triangleright A$ is the universal context which extends Γ and has a term of type A ”

$$\begin{array}{ccc} y(\Gamma \triangleright A) & \xrightarrow{\text{var}} & Tm \\ \uparrow \downarrow & \lrcorner & \downarrow ty \\ y\Gamma & \xrightarrow{A} & Ty \end{array}$$

WHAT IS A CwF?

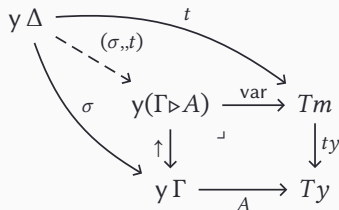
Read from the GAT definition! Or...

Definition (Fiore 2012, Awodey 2018)

A **natural model** consists of:

- a category $\mathcal{C}$ with a terminal object = contexts + substitutions + empty context
- two presheaves Ty and Tm (functors $\mathcal{C}^{op} \rightarrow \mathbf{Set}$) = types, terms, action of substitutions
- a natural transformation $ty : Tm \rightarrow Ty$ = maps a term to its type
- such that ty is locally representable = variables

“ $\Gamma \triangleright A$ is the universal context which extends Γ and has a term of type A ”



WHAT IS A CwF?

Read from the GAT definition

Definition (Fiore 2012)

A **natural model** consists

- a category $\mathcal{C}$ with
- two presheaves Ty
- a natural transformation
- such that ty is local

“ $T \triangleright A$ is the universal comonoid

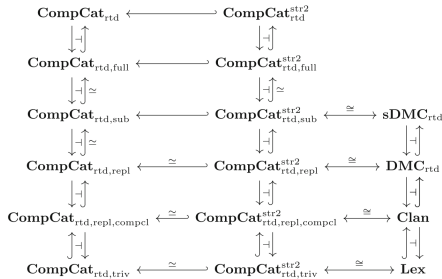


Fig. 4. Notions with types as maps, rooted

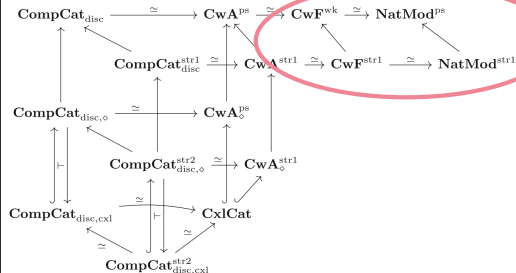


Fig. 5. Notions with types primitive

empty context
action of substitutions

SYNTAX

WHAT DO I MEAN BY SYNTAX?

Constructions that are valid in every model?

Syntax is the initial model

WHAT DO I MEAN BY SYNTAX?

Constructions that are valid in every model?

Syntax is the initial model

Some sort of manipulation of syntax trees?

WHAT DO I MEAN BY SYNTAX?

Constructions that are valid in every model?

Syntax is the initial model

Some sort of manipulation of syntax trees?

It is the same!

WHY ARE SYNTAX TREE NOT ENOUGH?

Ill-formed syntax trees: $0 + +$ $\lambda x.y$

Ill-typed syntax trees: $0 + (\lambda x.x)$

→ some syntax trees do not represent any valid terms

WHY ARE SYNTAX TREE NOT ENOUGH?

Ill-formed syntax trees: $0 + +$ $\lambda x.y$

Ill-typed syntax trees: $0 + (\lambda x.x)$

→ some syntax trees do not represent any valid terms

Equations: $\text{id} \circ f = f$

→ many syntax trees represent equal terms

WHY ARE SYNTAX TREE NOT ENOUGH?

Ill-formed syntax trees: $0 + +$ $\lambda x.y$

Ill-typed syntax trees: $0 + (\lambda x.x)$

→ some syntax trees do not represent any valid terms

Equations: $\text{id} \circ f = f$

→ many syntax trees represent equal terms

If f represents a valid term of type $\text{hom}(a, b)$, is $f \circ \text{id}_a$ valid?

→ equations play a role in typing!

Theorem (Cartmell 1986, Kaposi, Kovács and Altenkirch 2019)

The initial model of any GAT exists.

1. Start from syntax trees = “pre-terms”

Theorem (Cartmell 1986, Kaposi, Kovács and Altenkirch 2019)

The initial model of any GAT exists.

1. Start from syntax trees = “pre-terms”
2. (Mutually) define typing and definitional equality of pre-terms (predicates/relations)
= least relation closed under rules of the theory

Theorem (Cartmell 1986, Kaposi, Kovács and Altenkirch 2019)

The initial model of any GAT exists.

1. Start from syntax trees = “pre-terms”
2. (Mutually) define typing and definitional equality of pre-terms (predicates/relations)
= least relation closed under rules of the theory
3. Quotient the subset of well-typed pre-terms by definitional equality

Theorem (Cartmell 1986, Kaposi, Kovács and Altenkirch 2019)

The initial model of any GAT exists.

1. Start from syntax trees = “pre-terms”
2. (Mutually) define typing and definitional equality of pre-terms (predicates/relations)
= least relation closed under rules of the theory
3. Quotient the subset of well-typed pre-terms by definitional equality
4. This is a model, and it is moreover initial!

Theorem (Cartmell 1986, Kaposi, Kovács and Altenkirch 2019)

The initial model of any GAT exists.

1. Start from syntax trees = “pre-terms”
2. (Mutually) define typing and definitional equality of pre-terms (predicates/relations)
= least relation closed under rules of the theory
3. Quotient the subset of well-typed pre-terms by definitional equality
4. This is a model, and it is moreover initial!

Implementation: work with pre-terms, but check for typing/definitional equality

Much better if typing is **decidable**!

WRAPPING UP

One type theory to describe them all: a **logical framework**

Do the work **once**, get it for **all theories**:

- syntax
- (category of) models

Syntax is the **initial model**

WHERE TO GO FROM HERE?

To go on with GATs and CwFs

- *Generalised algebraic theories and contextual categories* (Cartmell 1986)
- *Categories with families: Unityped, simply typed, and dependently typed* (Castellan et al. 2021)
- *Constructing Quotient Inductive-Inductive Types* (Kaposi et al. 2019)
- *On generalized algebraic theories and categories with families* (Bezem et al. 2021)
- *Principles of Dependent Type Theory* (Angiuli and Gratzer 2026) (thanks for the preview!)

WHERE TO GO FROM HERE?

To go on with GATs and CwFs

- *Generalised algebraic theories and contextual categories* (Cartmell 1986)
- *Categories with families: Unityped, simply typed, and dependently typed* (Castellan et al. 2021)
- *Constructing Quotient Inductive-Inductive Types* (Kaposi et al. 2019)
- *On generalized algebraic theories and categories with families* (Bezem et al. 2021)
- *Principles of Dependent Type Theory* (Angiuli and Gratzer 2026) (thanks for the preview!)

Other logical frameworks

- SOGATs = GATs + built-in handling of contexts & variables
→ *Second-order generalised algebraic theories, by examples* (Kaposi 2025)
- a more syntactic version (// construction of syntax)
→ *A general definition of dependent type theories* (Bauer et al. 2020)
- proof assistants!
→ Isabelle, Dedukti, Andromeda 2, Beluga...

WHERE TO GO FROM HERE?

To go on with GATs and CwFs

- *Generalised algebraic theories and contextual categories* (Cartmell 1986)
- *Categories with families: Unityped, simply typed, and dependently typed* (Castellan et al. 2021)
- *Constructing Quotient Inductive-Inductive Types* (Kaposi et al. 2019)
- *On generalized algebraic theories and categories with families* (Bezem et al. 2021)
- *Principles of Dependent Type Theory* (Angiuli and Gratzer 2026) (thanks for the preview!)

Other logical frameworks

- SOGATs = GATs + built-in handling of contexts & variables
→ *Second-order generalised algebraic theories, by examples* (Kaposi 2025)
- a more syntactic version (// construction of syntax)
→ *A general definition of dependent type theories* (Bauer et al. 2020)
- proof assistants!
→ Isabelle, Dedukti, Andromeda 2, Beluga...

Theorem (Uemura 2022)

The category of GATs is the bi-initial category with finite limits and an exponentiable morphism.

Thank you!

BIBLIOGRAPHY I

- [GTW77] Joseph Goguen, James Thatcher and Eric Wagner. ‘An Initial Algebra Approach to the Specification, Correctness and Implementation of Abstract Data Types’. In: *Current Trends in Programming Methodology. Data Structuring*. 1977.
- [Car86] John Cartmell. ‘Generalised algebraic theories and contextual categories’. In: *Annals of Pure and Applied Logic* (1986). doi: 10.1016/0168-0072(86)90053-9.
- [Fio12] Marcelo Fiore. ‘Discrete generalised polynomial functors’. Talk at ICALP 2012. 2012. doi: 10.1007/978-3-642-31585-5_22.
- [Awo18] Steve Awodey. ‘Natural models of homotopy type theory’. In: *Mathematical Structures in Computer Science* 2 (2018). doi: 10.1017/S0960129516000268.
- [KKA19] Ambrus Kaposi, András Kovács and Thorsten Altenkirch. ‘Constructing Quotient Inductive-Inductive Types’. In: *Proceeding of the ACM on Programming Languages* POPL (Jan. 2019). doi: 10.1145/3290315.
- [BHL20] Andrej Bauer, Philipp G. Haselwarter and Peter LeFanu Lumsdaine. ‘A general definition of dependent type theories’. 2020. doi: 10.48550/arxiv.2009.05539. arXiv: 2009.05539 [math.LO].
- [Bez+21] Marc Bezem et al. ‘On generalized algebraic theories and categories with families’. In: *Mathematical Structures in Computer Science* 9 (2021). doi: 10.1017/S0960129521000268.
- [CCD21] Simon Castellan, Pierre Clairambault and Peter Dybjer. ‘Categories with families: Untyped, simply typed, and dependently typed’. In: *Joachim Lambek: The Interplay of Mathematics, Logic, and Linguistics* (2021).
- [Uem22] Taichi Uemura. ‘The universal exponentiable arrow’. In: *Journal of Pure and Applied Algebra* 7 (July 2022). doi: 10.1016/j.jpaa.2021.106991.
- [Kap25] Ambrus Kaposi. ‘Second-order generalised algebraic theories, by examples’. Lecture notes for the International School on Logical Frameworks and Proof Systems Interoperability. 2025.

- [Avr+26] Samy Avrillon et al. ‘For Generalised Algebraic Theories, Two Sorts Are Enough’. In: (27th Jan. 2026). doi: 10.48550/ARXIV.2601.19426. arXiv: 2601.19426 [cs.PL].
- [AG26] Carlo Angiuli and Daniel Gratzer. *Principles of Dependent Type Theory*. 2026. In preparation.